

# SIMATS ENGINEERING

## ASSESSMENT TOOL 1: Constraint-Based Problem Solving

**COURSE NAME:** Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

---

### COURSE OUTCOME COVERED

**CO5:** *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Assessment Tool Weightage: 40%

Dave's Taxonomy: Articulation (Level 4)  
Question Count: 5

**Total Marks: 50**

---

### Code of Conduct

I H.Mohammed Nawaz (192325060) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: H.Mohammed Nawaz (192325060)

### Assessment Tasks Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

| Criteria                     | Weightage | Level 4 - Exemplary                                      | Level 3 - Proficient        | Level 2 - Developing        | Level 1 - Beginning           |
|------------------------------|-----------|----------------------------------------------------------|-----------------------------|-----------------------------|-------------------------------|
| Understanding of Constraints | 25%       | All technical constraints are identified and prioritized | Most constraints identified | Some constraints identified | Constraints poorly understood |

## 1. Design an HDFS-Based Storage Solution for a Media Platform

A media platform generates a large amount of data such as videos, images, audio files, movies, and user-generated content. HDFS is suitable because it is designed to store large files across multiple machines. The main constraints are availability above 99.9%, replication factor not more than 3, and cost-efficient storage growth.

The proposed HDFS architecture consists of NameNode, Standby NameNode, and multiple DataNodes. The NameNode manages file names, block locations, and metadata, while DataNodes store the actual media blocks. A replication factor of **3** should be used for important media data. This means each block has three copies stored on different DataNodes.

To improve availability, NameNode High Availability should be configured with an Active NameNode and a Standby NameNode. If the Active NameNode fails, the Standby NameNode can take over. **Rack awareness** should also be enabled so that replicas are distributed across different racks. Therefore, failure of a single server or rack will not make the data unavailable.

For cost efficiency, the platform can use storage tiering. Frequently accessed videos can be stored on faster storage, while old or rarely accessed media can be moved to cheaper storage. Compression and appropriate HDFS block sizes can also reduce storage requirements.

Regular monitoring, health checks, backups, and capacity planning should be used to maintain reliability. Storage capacity should be increased gradually based on actual usage instead of purchasing excessive hardware in advance.

**Conclusion:** Using replication factor 3, NameNode HA, rack-aware storage, monitoring, compression, and storage tiering provides high availability while keeping the HDFS media platform cost-efficient and scalable.

## 2. MapReduce Workflow for Processing Log Data Within 20 Minutes

An organization needs to process a large amount of log data within a 20-minute batch window while using a limited number of cluster nodes. MapReduce is suitable because it divides a large processing task into smaller parallel tasks.

First, the log files should be stored in HDFS. HDFS divides large files into blocks and distributes them across available DataNodes. This allows multiple Mapper tasks to process different portions of the logs simultaneously.

The Mapper reads each log record and extracts useful information such as IP address, timestamp, URL, status code, or error type. For example, a Mapper can generate key-value pairs such as (status code, 1). Several Mapper tasks can run in parallel depending on the number of available nodes.

The Shuffle and Sort phase groups values having the same key. For example, all records with status code 404 are grouped together. The Reducer then calculates the final result, such as the total number of errors or requests.

A **Combiner** should be used where possible because it performs partial aggregation at the Mapper side. This reduces network traffic during the shuffle phase. Data locality should also be preferred so that computation happens close to the data.

Because the number of nodes is limited, resources should be divided carefully between Mapper and Reducer tasks. The job should be tested with sample data to estimate execution time and ensure that it remains below 20 minutes.

**Conclusion:** Parallel Mappers, combiners, optimized Reducers, data locality, and proper resource allocation can allow the MapReduce workflow to process the logs efficiently within the 20-minute batch requirement.

---

### 3. YARN Scheduler Policy for ETL, Reporting, and Machine Learning

YARN is responsible for managing CPU, memory, and other resources in a Hadoop cluster. In this organization, the cluster must support **ETL, reporting, and machine-learning jobs**. The scheduler must provide fairness, give priority to ETL, and recover from node failures.

A suitable approach is to use the **YARN Capacity Scheduler** with separate queues. For example, three queues can be created: ETL, Reporting, and Machine Learning. The ETL queue can receive the highest priority because ETL prepares data required by other applications. Reporting can receive medium priority, while machine-learning jobs can use a separate queue with controlled resources.

Fairness should be maintained by assigning resource limits to each queue. ETL should receive enough resources to complete quickly, but it should not permanently consume the entire cluster. Reporting and machine-learning jobs should continue receiving resources according to their queue capacity.

For example, the cluster could allocate approximately 50% resources to ETL, 25% to reporting, and 25% to machine learning, depending on organizational requirements. These values can be adjusted according to workload.

For node failure, YARN should detect unhealthy or failed nodes and stop assigning new containers to them. Failed tasks should be rescheduled on healthy nodes. Long-running machine-learning jobs should use checkpointing so that they can recover without restarting from the beginning.

Monitoring and alerts should be used to identify resource shortages and failed nodes.

**Conclusion:** A queue-based Capacity Scheduler with ETL priority, fair resource allocation, resource limits, monitoring, and automatic task recovery provides efficient and reliable YARN resource management.

---

#### 4. Enterprise Architecture for SAN, NAS, and Operational Databases

An enterprise needs to collect data from different sources such as **SAN, NAS, and operational databases** and move it into a big data platform. The architecture should provide reliable backup while avoiding unnecessary duplication.

The proposed architecture uses Apache NiFi as the data ingestion and flow-management layer. NiFi can receive data from different sources, route it, transform it, monitor its movement, and provide data-flow tracking.

Data from operational databases can be collected using database connectors or Change Data Capture (CDC) so that only changed records are transferred instead of repeatedly copying the entire database. Files from NAS and SAN can be transferred through controlled file-ingestion processes.

The collected data can be stored in **HDFS**. HDFS replication can provide fault tolerance, while important datasets can also be backed up to separate storage. A replication factor of up to 3 provides reliability without creating too many copies.

To reduce duplication, the system should use **unique record IDs, checksums, timestamps, and duplicate detection**. NiFi can also maintain flow information and prevent the same file or record from being unnecessarily processed multiple times.

Data can be organized into separate zones such as **raw, processed, and archived data**. Access controls and encryption should protect sensitive enterprise information.

**Conclusion:** Using SAN/NAS connectors, database ingestion, Apache NiFi, HDFS, replication, backup, and duplicate detection provides reliable and efficient enterprise data ingestion with minimum unnecessary duplication.

## 5. End-to-End Hadoop Ecosystem Solution

An end-to-end Hadoop solution should support secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. A suitable architecture is:

Data Sources → Apache NiFi → HDFS → YARN → MapReduce/Spark → Hive → Reports/Applications

Apache NiFi can collect data from databases, files, applications, and other sources. Data should be transferred using secure protocols such as HTTPS or SFTP, and encryption should be used to protect sensitive information during movement.

After ingestion, data can be stored in HDFS. A replication factor of up to 3 can be used to provide fault tolerance. HDFS should use rack awareness so that replicas are distributed across different physical racks.

For high availability, the Hadoop cluster should use NameNode High Availability with Active and Standby NameNodes. If the Active NameNode fails, the Standby NameNode can take over. DataNodes should also be monitored so that failed nodes can be detected and workloads can be moved to healthy nodes.

YARN manages cluster resources and distributes processing workloads. MapReduce or Spark can perform data processing, while Hive can be used for SQL-based analysis and reporting.

NiFi or an ETL pipeline can clean, transform, validate, and route data before it reaches the processing layer. Security can be improved using authentication, authorization, encryption, access controls, and audit logs.

**Conclusion:** The proposed Hadoop ecosystem provides secure data transfer through NiFi, reliable HDFS storage, NameNode high availability, YARN resource management, scalable data processing, and ETL integration. Therefore, it satisfies the requirements of security, availability, and efficient data processing.

### Constraint-Based Problem Solving Rubric

|                       |     |                                                        |                   |                        |             |
|-----------------------|-----|--------------------------------------------------------|-------------------|------------------------|-------------|
|                       |     | accurately                                             |                   |                        |             |
| System Design Quality | 25% | Design is robust, feasible, and aligned to constraints | Reasonable design | Basic design with gaps | Weak design |

|                                |     |                                                                  |                      |                         |                          |
|--------------------------------|-----|------------------------------------------------------------------|----------------------|-------------------------|--------------------------|
| Application of Hadoop Concepts | 20% | Uses HDFS, MapReduce, YARN, and integration concepts effectively | Mostly effective use | Partial use of concepts | Weak concept application |
| Justification                  | 15% | Strong technical reasoning for all choices                       | Good justification   | Basic justification     | Little justification     |
| Clarity                        | 15% | Clear, wellstructured, and professional answer                   | Mostly clear         | Somewhat unclear        | Poorly structured        |